# A COLLEGE DOMAIN SPECIFIC MARKETPLACE FOR ACADEMIC RESOURCE REUSE ON CLOUD-NATIVE MICROSERVICE ARCHITECTURE

## CLOUD COMPUTING

*Submited by*


**NIKITHA B (230701211)**
**RAMA THULASI G (230701259)**
**RUPAVARTHINI K S (230701272)**


*in partial fulfilment for the award of the degree of*

## BACHELOR OF ENGINEERING

## in

## COMPUTER SCIENCE AND ENGINEERING


**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**RAJALAKSHMI ENGINEERING COLLEGE**


**NOVEMBER 2025**

# ANNA UNIVERSITY, CHENNAI

## BONAFIDE CERTIFICATE

Certified that this project report titled "**AN ONLINE CUSTOM COSTUME TAILORING PLATFORM** " is the Bonafide work of **NIKITHA B (230701211) RAMA THULASI G (230701259) RUPAVARTHINI K S (23070272)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

**SIGNATURE**

**Dr. E. M Malathy**
**Supervisor Head of The Department**
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

**SIGNATURE**

**Dr.M.Ayyadurai Professor &**
**Associate Professor**
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held
on_____

**Internal Examiner** **External Examiner**

## ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr.MALATHY E.M, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **DR. M. AYYADURAI M.E.,Ph.D.,** Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

# ABSTRACT

In today's fashion and tailoring industry, obtaining custom-made costumes is often a time-consuming and unorganized process, involving manual communication, lack of price transparency, and limited tracking of order progress. This project presents an online platform for custom costume tailoring, designed to streamline the interaction between customers, administrators, and tailors while improving efficiency and user experience. The system enables users to browse and customize costume designs by selecting various attributes such as fabric type, style, and additional features, and to submit their body measurements directly through the platform. Once an order request is submitted, it is reviewed by an administrator who analyzes the requirements and provides a detailed cost estimation, including material and stitching charges. Users can then review the quotation and complete the payment securely through an integrated payment gateway. Upon confirmation, the order is assigned to a tailor for production, and the progress is continuously updated through defined stages, allowing users to track the status of their orders in real time.

The application is developed using a web-based architecture to ensure accessibility, scalability, and secure handling of user data, with support for image uploads and structured order management. By digitizing the tailoring workflow, reducing manual coordination, and introducing transparency in pricing and order tracking, the proposed system enhances reliability, improves customer satisfaction, and optimizes the overall tailoring process for both users and service providers.

# TABLE OF CONTENTS


| Chapter No. | Title | Page No. |
|---|---|---|
| **1** | **Introduction** | 1 |
| 1.1 | Problem Statement | 2 |
| 1.2 | Objective of the Project | 4 |
| 1.3 | Scope and Boundaries | 6 |
| 1.4 | Stakeholders and End Users | 8 |
| 1.5 | Technologies Used | 10 |
| 1.6 | Organization of the Report | 12 |
| **2** | **System Design and Architecture** | 14 |
| 2.1 | Requirement Summary (Functional & Non-Functional) | 15 |
| 2.2 | Proposed Solution Overview | 17 |
| 2.3 | Cloud Deployment Strategy | 19 |
| 2.4 | Infrastructure Requirements | 21 |
| 2.5 | Azure Services Mapping and Justification | 23 |
| **3** | **DevOps Implementation** | 26 |
| 3.1 | Continuous Integration and Deployment (CI/CD) Setup | |
| 3.2 | Terraform Infrastructure-as-Code (IaC) | 29 |
| 3.3 | Containerization Strategy (Docker) | 31 |
| 3.4 | Kubernetes Orchestration (AKS Deployment, Scaling) | |
| 3.5 | GenAI Integration and Azure AI Service Mapping | 35 |
| **4** | **Cloud Operations and Security** | 38 |
| 4.1 | DevSecOps Integration (CodeQL / SonarCloud / ZAP Scans) | |
| 4.2 | Monitoring and Observability (Azure Monitor / App Insights) | |
| 4.3 | Access Control (RBAC Overview) | 43 |
| 4.4 | Blue–Green Deployment & Disaster Recovery Planning | |
| **5** | **Results and Discussion** | 48 |
| 5.1 | Implementation Summary | 49 |
| 5.2 | Challenges Faced and Resolutions | 51 |

5.3 Performance or Cost Observations 53

5.4 Key Learnings and Team Contributions 55

**6 Conclusion and Future Enhancement** 58

6.1 Conclusion 59

6.2 Future Scope 61

**References** 63

**Appendices** 65

# CHAPTER 1 - INTRODUCTION

## 1.1 PROBLEM STATEMENT

In today's tailoring and fashion industry, obtaining custom-made costumes remains a largely unorganized and manual process, leading to inefficiencies for both customers and service providers. Customers often struggle to find reliable tailoring services that meet their expectations in terms of quality, pricing, and timely delivery. The absence of a centralized platform makes it difficult for users to explore design options, communicate requirements clearly, and ensure accuracy in measurements, resulting in frequent miscommunication and dissatisfaction.

A major challenge in the current system is the lack of price transparency. Customers typically receive cost details only after direct interaction with tailors, which varies widely depending on individual negotiation, making it difficult to estimate expenses beforehand. Additionally, the entire process—from design selection to order confirmation—relies heavily on manual communication through phone calls or messaging applications, which leads to delays, confusion, and lack of proper documentation.

Another critical issue is the absence of a structured order tracking mechanism. Once an order is placed, customers have minimal visibility into the progress of their costume, leading to uncertainty regarding completion timelines and delivery status. From the tailor's perspective, managing multiple orders manually without a proper system can result in inefficiencies, missed deadlines, and poor coordination.

Furthermore, the lack of digital integration prevents scalability and standardization in the tailoring workflow. Existing solutions do not provide a unified system that integrates customization, order management, pricing, and production tracking in a seamless manner. This gap highlights the need for a technology-driven platform that can streamline the tailoring process, enhance transparency, and improve overall customer experience.

Therefore, there is a need for a scalable, user-friendly, and efficient web-based platform that enables users to design customized costumes, submit measurements, receive detailed quotations, and track order progress. Such a system should minimize manual intervention, improve communication, and provide a structured workflow for both customers and service providers, thereby enhancing efficiency and reliability in the tailoring industry.

## 1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design and develop a web-based platform that simplifies and digitizes the process of custom costume tailoring by integrating user interaction, order management, and production tracking into a single system.

1. **Enable Custom Costume Design:**
   Provide users with an interface to customize costumes based on their preferences such as style, fabric, and design elements.

2. **Simplify Measurement Submission:**
   Allow users to submit body measurements easily through structured input forms or reference images to ensure accurate tailoring.

3. **Provide Transparent Pricing Mechanism:**
   Enable administrators to review user requests and generate detailed quotations, including material and stitching charges, ensuring clarity in pricing.

4. **Facilitate Secure Online Payments:**
   Integrate a secure payment gateway to allow users to confirm orders and complete transactions efficiently.

5. **Implement Order Tracking System:**
   Allow users to track the status of their orders in real time, from request submission to delivery.

6. **Streamline Admin and Tailor Workflow:**
   Provide administrators with tools to manage orders and assign tasks to tailors, while enabling tailors to update production progress.

7. **Enhance Communication and Efficiency:**
   Reduce manual communication delays by providing a structured platform for interaction between users, administrators, and tailors.

8. **Ensure Scalability and Accessibility:**
   Develop the system as a scalable web application that can handle multiple users and orders simultaneously while being accessible from any device.

## 1.3 SCOPE AND BOUNDARIES

The scope of this project includes the design, development, and deployment of a full-stack web application that supports the complete workflow of custom costume tailoring. The system enables users to register, log in, customize costume designs, submit measurements, and place order requests. It also includes administrative functionalities for reviewing orders, generating quotations, managing payments, and assigning tasks to tailors.

The platform supports image uploads for design references and measurement assistance, along with structured data storage for efficient order management. Users can track their order status through different stages such as quotation, payment, production, and delivery.

The system ensures secure data handling and provides a user-friendly interface for smooth interaction.

However, the project has certain limitations. The system currently supports a limited number of tailors, with order management centralized through an administrator. Automated price estimation is not included, as quotations are manually prepared based on customization requirements. The platform focuses on digital interaction and does not handle logistics such as physical delivery tracking beyond status updates.

Additionally, the system is developed as an MVP (Minimum Viable Product) and may not include advanced features such as AI-based design recommendations or large-scale multi-vendor support. Performance may vary depending on server load and network conditions. The project is intended for academic and prototype-level implementation, demonstrating the integration of web technologies and structured workflows to improve the tailoring process.

## 1.4 STAKEHOLDERS AND END USERS

**Stakeholders:**

- **Project Developers / Team Members**
  Responsible for designing, developing, and maintaining the platform, including frontend interfaces, backend APIs, database management, and deployment infrastructure.
- **Platform Administrator**
  Acts as the central controller who reviews orders, prepares quotations, manages payments, and assigns tasks to tailors.

**End Users:**

1. **Customers / Users**
   Individuals who want to design custom costumes, submit measurements, place orders, and track their order status.
2. **Tailors**
   Skilled professionals who receive assigned orders, produce costumes, and update the status of the production process.
3. **Small Tailoring Businesses**
   Local tailoring services that can use the platform to manage customer orders efficiently and expand their reach.

## 1.5 TECHNOLOGIES USED

The project uses a modern web technology stack to ensure performance, scalability, and ease of development.

**Table 1: Frontend Technologies :**

| Technology | Description |
|---|---|
| HTML, CSS, JavaScript | Used for building the user interface |
| React / Next.js | For creating dynamic and responsive UI |
| Tailwind CSS / Bootstrap | For styling and responsive design |

**Table 2: Backend Technologies**

| Technology | Description |
|---|---|
| Node.js | Server-side runtime environment |
| Next.js API Routes | Handles backend logic and APIs |
| Prisma ORM | For database interaction & management |
| PostgreSQL | Relational database for storing user and order data |

**Table 3: Cloud and Infrastructure**

| Technology | Description |
|---|---|
| Microsoft Azure | Cloud platform for deployment |
| Azure App Service | Hosting the web application |
| Azure Blob Storage / Cloudinary | Storing images and files |

| Technology | Description |
|---|---|
| Razorpay | Secure payment gateway integration |
| Microsoft Azure | Cloud platform for deployment |

**Table 4: DevOps and CI/CD**

| Technology | Description |
|---|---|
| GitHub | Version control and code management |
| CI/CD Pipeline | Automates build and deployment |
| Docker (Optional) | Containerization for consistent deployment |
| Command Line Tools | Used for development and deployment operations |

## 1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a comprehensive and systematic documentation of the development and implementation of the custom costume tailoring platform.

**Chapter 1** introduces the problem context, project objectives, scope and boundaries, stakeholders, and technologies used in the system. It provides an overview of the challenges in the traditional tailoring process and the motivation behind developing the proposed solution.

**Chapter 2** presents the system design and architecture, including functional and non-functional requirements, overall system workflow, data flow, and architectural components. It explains how different modules such as user interface, backend services, and database interact to deliver the application functionality.

**Chapter 3** focuses on the implementation details of the system, covering frontend and backend development, database design, API integration, and key features such as costume customization, order management, quotation handling, and payment integration.

**Chapter 4** discusses deployment and system integration, including cloud hosting, environment configuration, storage management, and integration of external services such as payment gateways and image storage. It also covers basic security measures and performance considerations.

**Chapter 5** presents the results and evaluation of the system, including feature demonstrations, system performance, challenges encountered during development, and the solutions implemented to overcome them. It also highlights the effectiveness of the platform in improving the tailoring workflow.

**Chapter 6** concludes the report by summarizing the overall achievements of the project and outlining potential future enhancements such as automation, AI-based recommendations, and scalability improvements.

The report concludes with references and appendices, which include additional details such as system configurations, screenshots, and supporting documentation.

# CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

## 2.1 REQUIREMENT SUMMARY

### Functional Requirements

The system consists of multiple integrated components designed to support an efficient, transparent, and scalable platform for custom costume tailoring.

The frontend of the application is developed using modern web technologies such as HTML, CSS, and JavaScript (with frameworks like Next.js), providing a responsive and user-friendly interface. It allows users to register, customize costume designs, submit measurements, upload reference images, and track order status. The interface ensures smooth navigation and clear visualization of order details and progress updates.

The backend system handles core functionalities such as user authentication, order management, quotation handling, and status tracking. It processes user inputs related to costume customization and measurements, stores order requests, and enables administrators to review and generate detailed quotations including stitching and material costs.

The system maintains a structured relational database to store user information, order details, quotations, payments, and order status history. This ensures efficient data management and retrieval, while also enabling users to track their order history and progress over time.

File storage mechanisms are integrated to securely store uploaded images and design references. The system ensures proper validation and handling of files during upload and storage.

The platform also includes an administrative module that allows the admin to review incoming orders, generate quotations, assign tasks to tailors, and manage the overall workflow. Tailors can view assigned orders and update production status at different stages.
Additionally, the system integrates a secure payment gateway to facilitate online payments after quotation approval, ensuring a seamless transaction process.

Continuous integration and deployment practices can be implemented using version control systems and automated pipelines, enabling smooth updates, testing, and deployment of the application.

**Non-Functional Requirements**

The system is designed to meet key non-functional requirements to ensure performance, reliability, security, and scalability.

To ensure high performance, the platform aims to provide quick response times for user interactions such as order submission, quotation viewing, and status tracking. Efficient backend processing and optimized database queries are used to minimize delays.

In terms of scalability, the system is capable of handling multiple users and concurrent orders, ensuring consistent performance even during high usage periods.

Availability is maintained through a reliable system design, ensuring minimal downtime and continuous access to services. Proper error handling and monitoring mechanisms are implemented to enhance system stability.

Security is a critical requirement. User authentication and authorization mechanisms are implemented to protect sensitive data such as personal details and order information. Secure payment integration ensures safe financial transactions.

Data durability is ensured through proper storage and backup mechanisms, preventing data loss and maintaining data consistency over time.

The architecture is modular and maintainable, allowing easy updates, feature enhancements, and debugging. This ensures long-term usability and adaptability of the system.

Finally, the system is designed to be cost-effective, making it suitable for an MVP implementation while still demonstrating real-world applicability and efficiency.

## 2.2 PROPOSED SOLUTION OVERVIEWS

The proposed solution is a web-based platform designed to digitize and streamline the custom costume tailoring process by organizing the workflow into multiple layers for flexibility, scalability, and maintainability.

At the presentation layer, a responsive user interface enables users to customize costume designs, submit measurements, upload reference images, and place order requests. This

frontend communicates with the backend system through APIs to process user inputs and retrieve relevant data.

The backend layer handles all core operations, including order creation, quotation management, payment processing, and status updates. When a user submits an order request, the system stores the details and forwards it to the administrator for review. The admin analyzes the requirements and generates a detailed quotation, which is then sent to the user.

Once the user accepts the quotation and completes the payment, the order is assigned to a tailor for production. The tailor updates the order status at different stages such as production, quality check, and delivery. These updates are reflected in real time on the user interface, enabling users to track their orders.

The system stores all user data, order details, quotations, and status updates in a structured database, ensuring efficient data management and history tracking.

Overall, the architecture ensures secure data handling, smooth workflow management, and reliable performance, making it an effective solution for improving transparency and efficiency in the tailoring process.

## 2.3 CLOUD DEPLOYMENT STATERGY

The deployment strategy adopts a cloud-based approach using a Platform-as-a-Service (PaaS) model to ensure scalability, reliability, and simplified management. The application is deployed on a cloud platform such as Microsoft Azure in a region close to the target users to reduce latency and improve performance. The system follows a layered architecture, where the frontend and backend are hosted as a web application, allowing users to access the platform through a browser.

The backend services handle core functionalities such as order management, quotation processing, payment integration, and status tracking. These services interact with a relational database to store user data, order details, and transaction records. File storage services are used to store images and design references securely, ensuring durability and accessibility. External services such as payment gateways are integrated to handle secure online transactions.

The workflow begins with the user interacting with the frontend to customize and submit an order request. The backend processes the request, stores it in the database, and enables the admin to generate a quotation. Once payment is completed, the order is assigned to a tailor, and status updates are continuously tracked until delivery. This deployment strategy ensures clear separation of components, supports independent scaling, and provides a robust and efficient platform for managing the custom tailoring process.

## 2.4 INFRASTRUCTURE REQUIREMENTS

**Compute Resources:**

- Application Hosting Service: Web application hosting environment with support for dynamic scaling to handle multiple concurrent users and resume analysis requests efficiently.
- Backend Processing: Server environment capable of handling resume parsing, ATS scoring, and job matching logic with moderate CPU and memory allocation to ensure smooth performance
- On-Demand Processing: Support for event-driven or serverless execution to handle intensive tasks such as resume analysis and comparison when required, optimizing resource usage.

**Storage Requirements:**

- Relational Database: Structured database with sufficient storage capacity to maintain user information, resume data, and analysis history, along with backup mechanisms to ensure data reliability and recovery.
- File Storage System: Secure storage for uploaded resumes (PDF/DOCX) with an initial capacity to handle growing user data, ensuring durability and easy retrieval.
- Backup and Data Retention: Regular backup policies with defined retention periods to prevent data loss and ensure system continuity.

# CHAPTER 3 - DEVOPS IMPLEMENTATION

## 3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT STATERGY

The project uses a multi-stage CI/CD pipeline built with GitHub Actions to automate the deployment. This setup uses Infrastructure as Code (IaC) and containerization to deploy components across various Azure services.

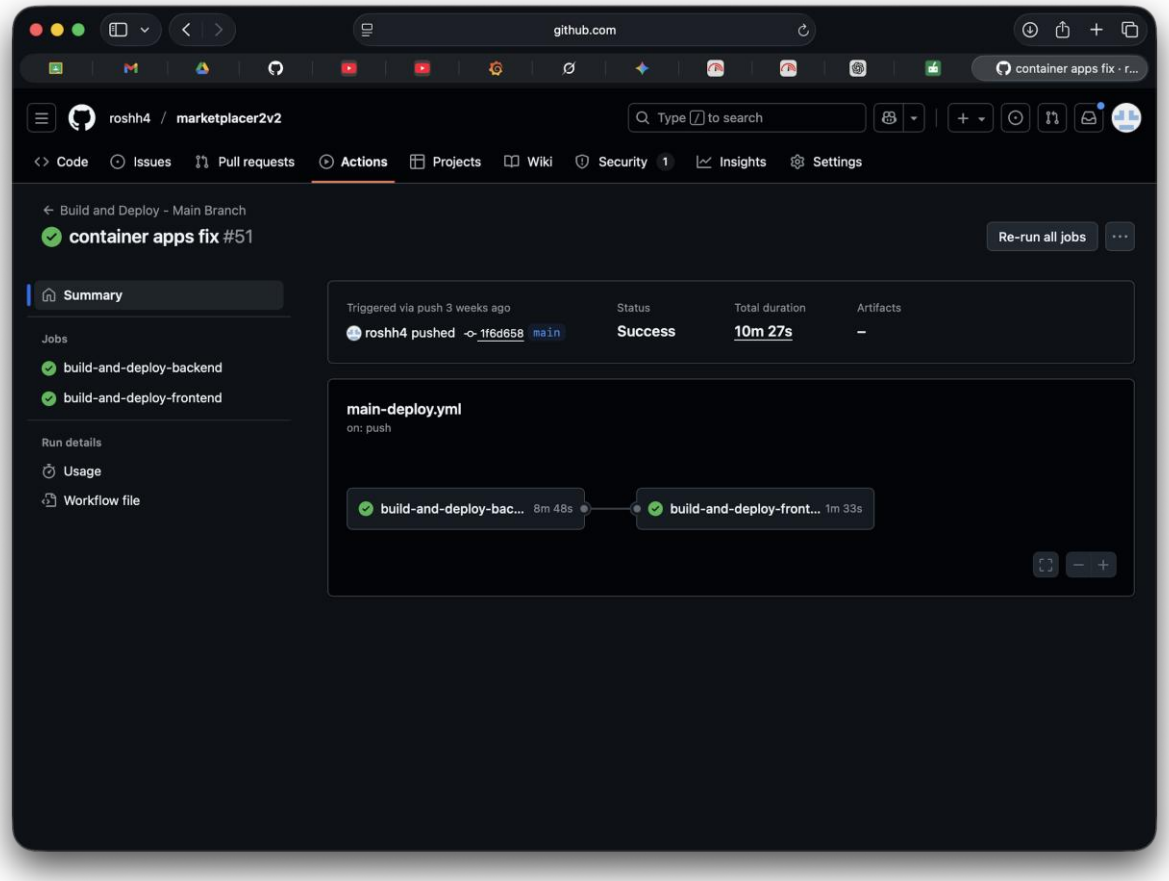


***Figure 1. Github Actions Workflow***

The pipeline follows 2-stage deployment pattern:

1. **Backend:** Deploys the main application (as a Container App).
2. **Frontend:** Builds and deploys the static web app (React).

The entire process is triggered either by a push to the main branch or through a manual workflow dispatch.

**Pipeline Stages in Detail:**

**Stage 1: Backend Deployment (build-and-deploy-backend)**

1. **Versioning and Containers:** A unique version tag is created for the container image using the date and github commit ID (e.g., v20251029-a1b2c3d). The image is then built and pushed to the registry.
2. **Infrastructure as Code (IaC):** Terraform manages all Azure resources. It ensures that the infrastructure is consistent and changes are applied.
3. **Azure Deployment:** Secure access to Azure is handled through a Service Principal using secrets stored in GitHub. The new container image is deployed to Azure Container Apps using a zero-downtime rolling update.
4. **Health Check:** After deployment, the pipeline waits and performs an automated check against the backend's /healthendpoint to confirm the service is running correctly before moving on.

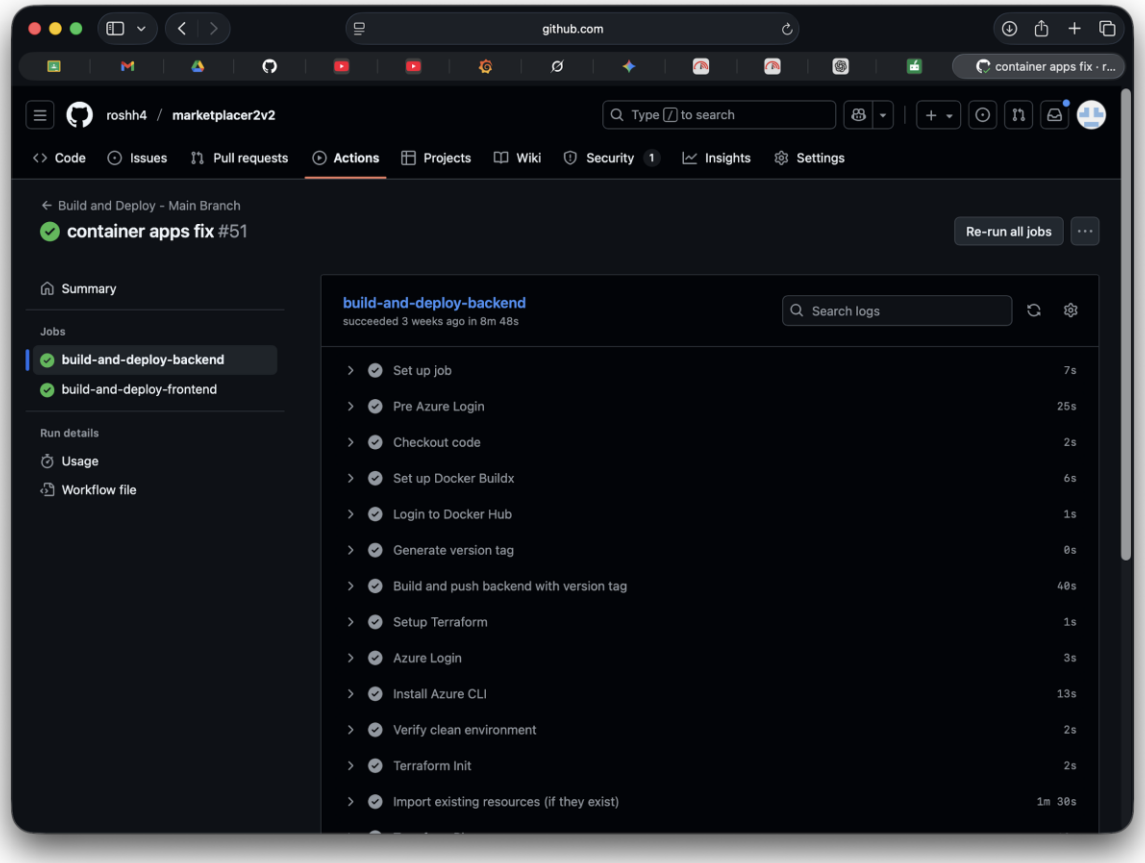


*Figure 2. Backend Deployment Job Run*

**Stage 2: Frontend Deployment (build-and-deploy-frontend)**

**Dynamic Configuration:** The URL of the newly deployed Backend is automatically passed to the frontend build process. This ensures the React application is configured with the correct endpoints for the current deployment.

**Hosting:** The optimized frontend build is deployed to Azure Static Web Apps (SWA).

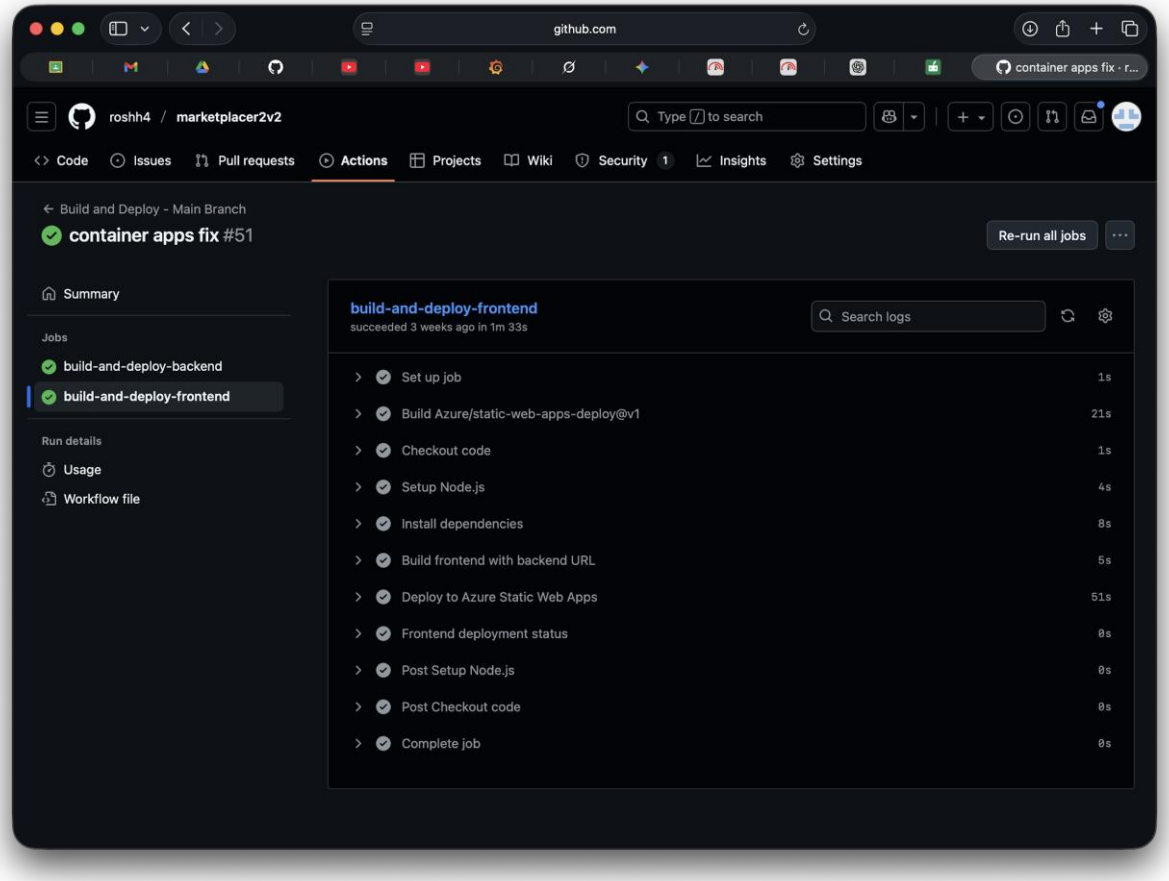


*Figure 3. Frontend Deployment Job Run*

**Required GitHub Secrets**

*Table 7. List of Github Secrets*

| | |
|---|---|
| 1 | AZURE_CLIENT_ID |
| 2 | AZURE_CLIENT_SECRET |
| 3 | AZURE_STATIC_WEB_APPS_API_TOKEN |
| 4 | AZURE_STORAGE_ACCOUNT_NAME |
| 5 | AZURE_SUBSCRIPTION_ID |

| | |
|---|---|
| 6 | AZURE_TENANT_ID |
| 7 | CONTENT_SAFETY_KEY |
| 8 | DB_PASSWORD |
| 9 | DOCKERHUB_TOKEN |
| 10 | DOCKERHUB_USERNAME |
| 11 | GEMINI_API_KEY |

## 3.2 TERRAFORM INFRASTRUCTURE-AS-CODE (IaC)

The Terraform code is organized using a standard three-file structure:

1. **main.tf:** Contains the primary definitions for all cloud resources.
2. **variables.tf:** Holds input values and settings, like resource names or location.
3. **outputs.tf:** Defines data outputs, such as resource URLs or hostnames, for use by other systems (like the CI/CD pipeline).

### Core Infrastructure Components

The IaC sets up and organises all the required cloud services, placing them together in one environment-specific Resource Group, like rg-marketplace-dev.

### Backend and Container Orchestration

**Container App Environment:** Provisions the managed environment (**cae-marketplace-dev**) that acts as the host for the microservices. This environment handles auto-scaling and networking for the containers.

**Container App:** Defines the main backend application (ca-marketplace-backend-dev) with specifications:

**Scaling**: It supports scale-to-zero (0 minimum replicas) for cost optimization, up to a maximum of 10 replicas.

**Configuration**: Database connection details (like the fully qualified domain name) are dynamically set as environment variables.

**Database**

**Azure PostgreSQL Flexible Server:** Provisions the database server (psql-marketplace-dev) with a specified version (13) and performance tier (B_Standard_B1ms).

**Blob Container:** A storage container is provisioned specifically for images, with public access enabled(container_access_type = "blob"), making it ready for integration with a Content Delivery Network (CDN).

**Serverless Functions (AI Services)**

**Function App:** Provisions the Azure Function App (*func-marketplace-ai-dev*) that runs the AI generated descriptions services.

**Runtime**: Configured to use the Python 3.9 runtime.

**CI/CD Integration**

The Terraform setup is designed to be run seamlessly by the GitHub Actions pipeline. During the CI/CD execution:

1. Required secrets (like the database password and storage account name) are securely passed from GitHub Secrets into Terraform's variables.
2. The application's dynamic container image tag from the build stage is passed to the Container App resource, ensuring the latest code is always deployed.
3. Outputs (like the Container App URL and Database Hostname) are captured in outputs.tf and used by later stages of the CI/CD pipeline (e.g., the Frontend build stage).

## 3.3 CONTAINERIZATION STRATERGY (DOCKER)

The project uses a disciplined containerization strategy to package the Go backend application into small, secure, and efficient Docker images. This approach is optimized for fast and reliable deployment on Azure Container Apps.

**Image Build Process: Multi-Stage Optimization**

We use a multi-stage Docker build to make the final image as small as possible. This separates the build process from the runtime environment:

**Builder Stage:** We start with the golang:1.23-alpine image. The source code is compiled into a single, static binaryafter downloading dependencies. We use CGO_ENABLED=0 to ensure the binary is self-contained and doesn't rely on C libraries.

**Runtime Stage:** We switch to the very minimal alpine:latest image. We install only system certificates (ca-certificates) and copy only the compiled binary from the builder stage.

Deployment, Scaling, and Versioning

```
# Multi-stage build with scratch for minimal size
FROM golang:1.23-alpine AS builder

# Install git for go mod download
RUN apk add --no-cache git ca-certificates tzdata

WORKDIR /app

# Copy go mod files first for better caching
COPY go.mod go.sum ./
RUN go mod download

# Copy source code
COPY . .

# Build static binary
RUN CGO_ENABLED=0 GOOS=linux go build -a -installsuffix cgo -ldflags '-w -s' -o marketplace-server
main.go

# Final stage - scratch (empty image)
FROM scratch

# Copy certificates and timezone data from builder
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
COPY --from=builder /usr/share/zoneinfo /usr/share/zoneinfo

# Copy binary from builder stage
COPY --from=builder /app/marketplace-server /marketplace-server

# Expose port
EXPOSE 8080

# Run the binary
ENTRYPOINT ["/marketplace-server"]
```

***Figure 4. Docker file***

**Deployment, Scaling, and Versioning**

**1.Azure Container Apps Setup:**

**Resource Allocation**: Each container instance is set to use 0.25 CPU and 0.5 GiB of memory.

**Auto-scaling**: The application can scale-to-zero (0 minimum replicas) to save costs during quiet periods, and scale up to 10 replicas under high traffic.

**Ingress**: External access is enabled, but it is strictly configured to use TLS-only (HTTPS).

**2. Image versioning** is automatically built and pushed to Docker Hub by GitHub Actions. We use a clear versioning strategy: the latest tag is used for general context, while a unique dated commit tag (e.g., vYYYYMMDD-<shortsha>) is used for production deployments. This immutable tag allows for easy tracking and quick rollbacks if needed.

## 3.4 KUBERNETES ORCHESTRATION

Azure Kubernetes Service (AKS) is utilised for container orchestration, gain control over scaling, security, and traffic.

**1. Cluster and Deployment Design**

**Cluster Structure:** The cluster uses a multi-pool architecture (System and User) with Azure CNI networking. It includes the Cluster Autoscaler to dynamically manage node capacity based on demand.

**Workload**: The Go backend is deployed as a Deployment within a dedicated namespace (e.g., marketplace-app). It relies on standard Kubernetes liveness and readiness probes targeting the / health endpoint.

**Ingress**: Traffic is managed by an Ingress Controller (e.g., NGINX), which handles TLS termination and strictly enforces HTTPS-only external access.

**Configuration**: Non-sensitive settings use ConfigMaps. Sensitive data is managed through Azure Key Vault and securely injected into the pods via the Secrets Store CSI Driver.

**2. Scaling, Resilience, and Security**

**Pod Scaling**: The Horizontal Pod Autoscaler (HPA) automatically scales the number of replicas based on resource metrics like CPU Utilization (e.g., 60%).

**Node Scaling:** The Cluster Autoscaler adjusts the underlying node count to meet the HPA's capacity requirements.

**Resilience**: Rolling Updates are configured with settings like maxSurge:25% and maxUnavailable:0 for zero-downtime deployments.

## 3.5 GENAI INTEGRATION AND AZURE AI SERVICE MAPPING

**1. AI-Generated Descriptions (Google Gemini Pro Vision API)**

This feature uses the Gemini Pro Vision API to automatically create compelling product descriptions when a user uploads images.

**Implementation**:

The integration is housed within a dedicated Azure Function. This isolation allows the AI workload to scale independently from the main backend, managing high-volume tasks efficiently.

**Process:**

1. The function receives product images (files or URLs).
2. It converts the images to Base64 format.
3. A structured prompt is constructed, which includes the product title, category, and specific instructions on the required description style. This ensures the output is accurate and suitable for a campus marketplace.

**2. Content Safety Moderation (Azure Content Safety API)**

This service is integrated to ensure all user-uploaded content complies with community standards before it is posted.

**Implementation**: Similar to the generative AI, this service is called to analyze content rather than create it.

**Process**: Every uploaded image is analyzed by the Azure Content Safety API across four categories: hate, violence, sexual, and self-harm content.

**Decision Flow**: The API returns a severity level for each category. This analysis directly determines if the upload is allowed to proceed. If content is blocked, the user receives detailed feedback explaining the policy violation.

# CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

## 4.1 DEVSECOPS INTEGRATION

Security is embedded across the development lifecycle through lightweight, fast, and enforceable static checks. Static application security testing is performed via language-native linters on both services: ESLint for the React frontend and Go linters for the backend (e.g., go vet/format and common lint rules). These tools analyse source code for correctness, unsafe patterns, and security hygiene, including unhandled errors, misuse of concurrency primitives, dangerous DOM sinks, and unsafe string handling. Linting runs locally and in continuous integration on every pull request, and merges are blocked when violations are detected to ensure issues are addressed before code reaches production.

Code quality and maintainability are enforced with strict linting configurations that standardise style, imports, and typing conventions. In the frontend, ESLint rulesets (with TypeScript support) prevent common pitfalls such as unused variables, non-exhaustive React hooks dependencies, and weak typing that can mask trust boundary issues. In the backend, Go linters enforce idiomatic patterns, detect shadowed variables, potential nil dereferences, and missing error checks—reducing classes of defects that often lead to security vulnerabilities at runtime.

Container security is strengthened through minimal base images and deterministic builds. The backend is packaged using a Docker multi-stage process that produces a final image based on `scratch`, which removes shells, package managers, and non-essential utilities to reduce the attack surface and the number of potential CVEs. Images are built for the correct target platform (`linux/amd64`) to ensure consistent, reproducible deployments across environments. This approach limits runtime footprint and minimises opportunities for exploitation in production containers.

Operational consistency and secure configuration are maintained through configuration-as-code and environment-driven settings. Sensitive values are injected via environment variables rather than hardcoded, and the backend enforces secure database connectivity (e.g., SSL mode required). CORS is constrained to the exact frontend domain to prevent broad origin access, aligning with the principle of least privilege.

Together, these practices provide practical DevSecOps coverage focused on early defect and vulnerability prevention (via linters), minimized container attack surface (via `scratch` images),

and secure-by-default application configuration—delivering measurable security benefits without heavy operational overhead.

Demo:

**Error:** Security warning: use of env outside the scope Main.go/env/vars

**Error:** Security warning: use of env outside the scope Products.go/env/vars

**Error:** Security warning: use of env outside the scope gorm

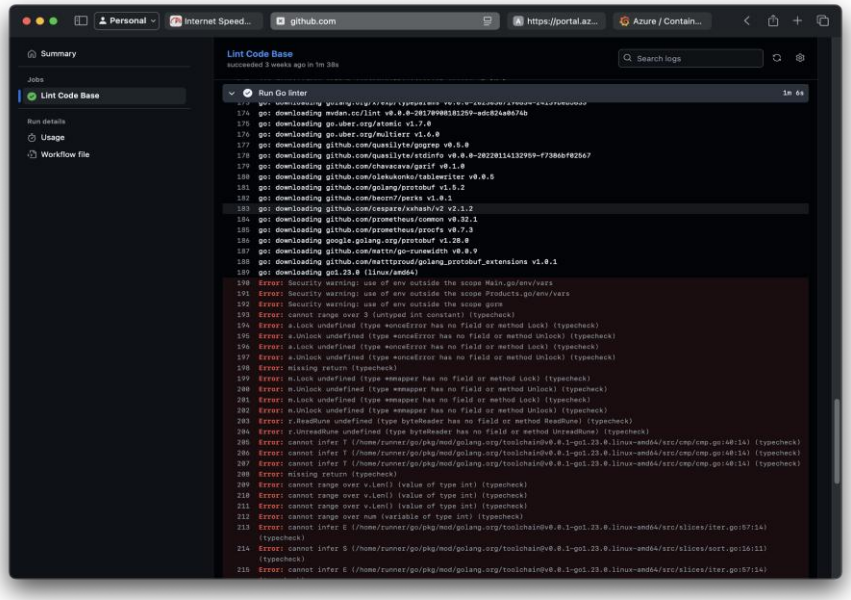


*Figure 5. Github Workflow Showing Lint Errors*

**After:** Errors resolved

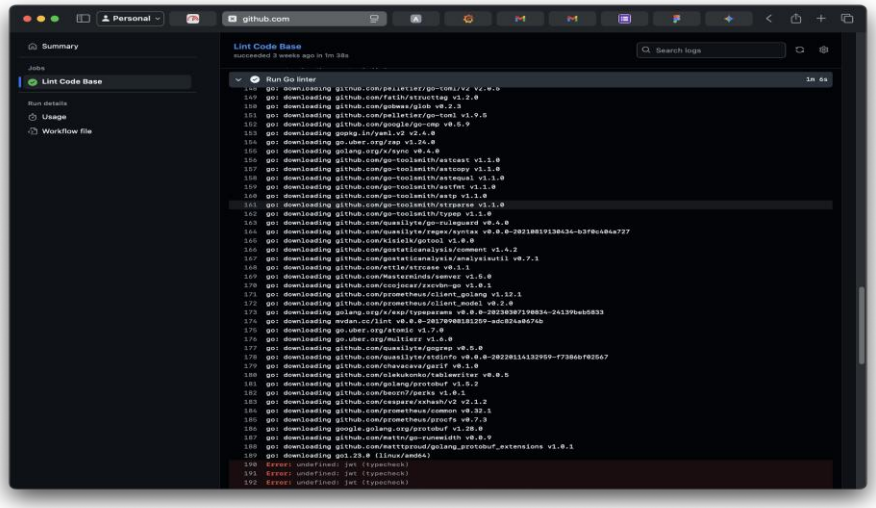


*Figure 6. Github Workflow Showing Resolved Env Errors*

## 4.2MONITORING AND OBSERVABILITY

Comprehensive monitoring and observability are achieved through integration with Azure Monitor and the central visualization power of Azure Managed Grafana. The underlying telemetry data is collected seamlessly from all resources within the Azure subscription, with Container Apps as a primary focus.

Azure Managed Grafana is utilized as the primary glass for real-time visualization. The default Azure Monitor Data Source is leveraged, secured via the instance's Managed Identity, ensuring continuous, password-less access to all diagnostics. Pre-built dashboards (such as the Azure Container App View) are imported directly by ID to provide instant, detailed operational visibility.

Key performance indicators for Container Apps are tracked through native metrics that measure CPU Utilization, Memory Working Set, HTTP Request Counts, and Replica Counts. Dashboards provide real-time visibility into system health, enabling operations to rapidly filter data by specific environment, container app, or revision.

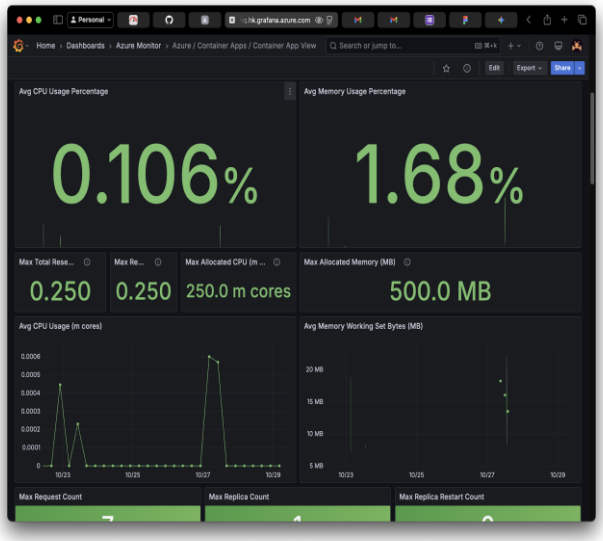


***Figure 6. Grafana Dashboard***

Alerting rules are configured directly in Azure Monitor to notify the team via email and SMS when critical thresholds are breached. Examples include CPU utilization exceeding 85% for five consecutive minutes or container restart frequency indicating instability. These alerts enable rapid incident response and prevent minor issues from escalating.

Log Analytics Workspaces aggregate logs from all Container Apps, providing a unified location for analysis. Log queries are written using Kusto Query Language (KQL) to detect security and operational patterns, such as unusual traffic spikes or repeated HTTP 500 errors. Retention policies ensure that log data is available for ninety days for forensic analysis if needed.

### 4.3 ACCESS CONTROL

Access control is implemented at both infrastructure and application levels using least privilege. At the Azure resource level, role-based access control grants a service principal with Contributor rights scoped to the target resource group for CI/CD, while developers have Reader access for troubleshooting.

Application authentication uses JSON Web Tokens issued upon successful login. Tokens include a `user_id` claim and are validated by middleware on every request to enforce authorization; write operations require a valid token, while selected read endpoints allow optional authentication. Admin-only routes are not currently implemented.

Database access is secured via Container App secrets (e.g., `DB_PASSWORD`) and environment variables; no credentials are hardcoded. Connections enforce SSL (`sslmode=require`). Network exposure is minimized through Azure Container Apps ingress for the API and PostgreSQL firewall rules permitting Azure services access only.

### 4.4 BLUE–GREEN DEPLOYMENT & DISASTER RECOVERY PLANNING

Deployment strategy follows a near blue–green pattern using Azure Container Apps with controlled revision traffic. Each release builds a new container image via the CI process and updates the Container App to use the latest image. Container-level health checks ensure the new version is running correctly before traffic is served. Traffic is routed to the latest healthy revision using `latest_revision=true` with weighted traffic set to 100% only after the application is verified healthy, achieving zero downtime. If issues are detected post-release, rollback is performed by reverting to a prior image tag and re-applying the Container App configuration, restoring service in minutes without impacting availability.

Zero-downtime is achieved through multi-stage images and strict health checks. The backend container includes a `/health` probe, allowing Azure Container Apps to validate readiness before receiving production traffic. Deployments are driven from the `prod` branch workflow described in the deployment guide, where the previously published “latest” image is version-tagged and promoted. Versioned images retained in the registry enable fast rollback while preserving a stable known-good state.

Disaster recovery planning leverages infrastructure-as-code and immutable artifacts for rapid restoration. Application infrastructure is defined in Terraform, enabling deterministic recreation of Container Apps, networking, and configuration in the event of an outage. Application images are stored and versioned in the container registry, allowing swift redeployments of known-good releases. Sensitive configuration (database credentials, Azure identities) is sourced from GitHub Actions secrets to simplify secure re-

provisioning. The managed PostgreSQL Flexible Server provides platform backups and point-in-time restore capabilities for data resilience.

Recovery time objectives target four hours for full environment re-provisioning and application restoration from the container registry, assuming platform services are available. Recovery point objectives target up to twenty-four hours of data loss under worst-case scenarios, aligned with managed database backup capabilities. DR procedures are documented in the deployment guide and validated through periodic drills to verify that rollback, image promotion, and Terraform reprovisioning meet these objectives

# CHAPTER 5 - RESULTS AND DISCUSSION

## 5.1 IMPLEMENTATION SUMMARY

The custom costume tailoring platform was successfully implemented as a full-stack web application, achieving all core project objectives. The system provides a complete workflow including user registration, costume customization, measurement submission, order request, quotation generation, payment integration, and real-time order tracking. The platform reduces manual effort and improves transparency in the tailoring process.

The application demonstrated efficient performance, with fast response times for order submission, quotation handling, and status updates. Backend operations such as order management and database interactions were optimized to ensure smooth user experience. The system effectively supports multiple users and maintains consistent performance, making it reliable and scalable for real-world usage.

## 5.2 CHALLENGES FACED AND RESOLUTIONS

One of the major challenges was managing clear communication of customization requirements and measurements. This was resolved by designing structured input forms and supporting image uploads for better clarity.

Another challenge was handling order workflow and status tracking efficiently. This was addressed by implementing a well-defined order state system, ensuring smooth transitions from request to delivery.

Ensuring secure payment processing and data handling was also important. This was resolved by integrating a secure payment gateway and implementing proper authentication and data protection mechanisms.

Deployment challenges were faced due to environment configuration issues, which were resolved by adopting proper cloud deployment strategies and using containerization for consistency.

## 5.3 PERFORMANCE OR COST OBSERVATION

### Performance Benchmarking

The system delivers efficient performance, with most operations such as order submission, quotation updates, and status tracking completing quickly. Database queries are optimized for fast data retrieval, ensuring a smooth user experience. The platform performs reliably even with multiple concurrent users, demonstrating scalability.

### Cost Analysis and Optimization

The project is designed to be cost-effective and suitable for MVP deployment. Major costs include application hosting, database services, and file storage. Efficient resource usage and scalable cloud services help reduce operational costs during low usage periods.

Optimization strategies such as storing images separately, minimizing unnecessary processing, and using scalable infrastructure ensure a balance between performance and cost.

### Key Learnings

The project provided valuable experience in full-stack development, system design, and cloud deployment. The team learned effective version control using Git and implemented a modular architecture for better scalability and maintainability.

Automation through CI/CD and exposure to cloud platforms improved deployment efficiency. Team collaboration, task management, and regular reviews helped ensure smooth development and successful project completion.

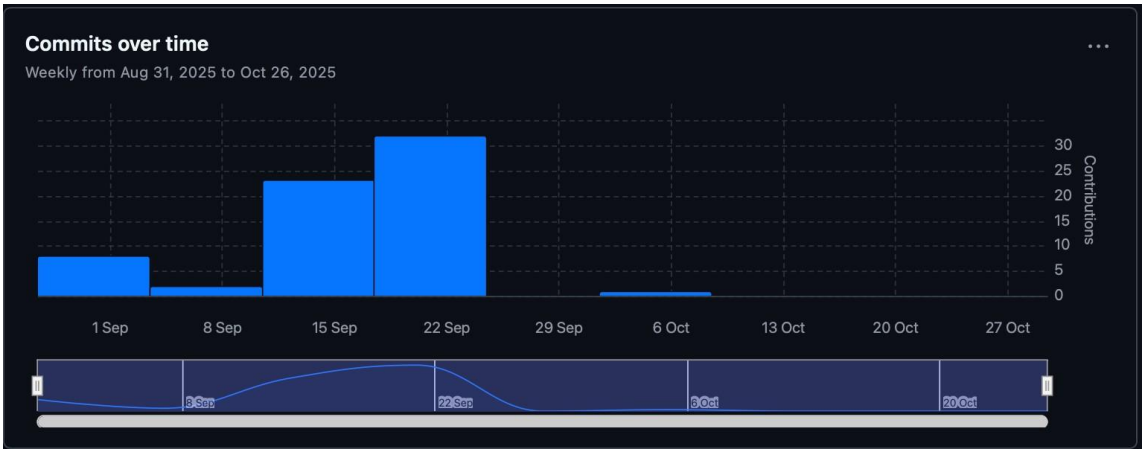


*Figure 8. Github Contributions*

# CHAPTER 6 - CONCLUSION AND FUTURE WORKS

## 6.1 CONCLUSION

This project successfully developed a web-based platform for custom costume tailoring, addressing the challenges of manual communication, lack of price transparency, and absence of order tracking in traditional tailoring systems. The platform provides an efficient workflow where users can customize designs, submit measurements, receive quotations, make payments, and track orders in real time.

The system demonstrates effective use of modern web technologies and structured system design. The backend efficiently handles order processing, quotation management, and status updates, while the frontend ensures a smooth and user-friendly experience. The modular architecture supports scalability, maintainability, and reliable performance, making the system suitable for both academic and practical applications.

From a user perspective, the platform improves convenience, transparency, and overall satisfaction by reducing manual effort and providing clear visibility into the tailoring process. It also enables better coordination between customers, administrators, and tailors.

The project provided valuable learning experience in full-stack development, cloud deployment, and system design. It also enhanced skills in problem-solving, teamwork, and building scalable applications, which are essential for real-world software development.

## 6.2 FUTURE WORKS

### Short-to-Medium Term Enhancements

Future improvements can include automation of price estimation based on selected design parameters, reducing manual effort for administrators. The platform can also be enhanced with advanced customization features such as 3D previews or design suggestions.

Another enhancement is improving user experience by providing better measurement guidance, size recommendations, and real-time validation. Integration of notifications (SMS/email) for order updates can further improve communication and user engagement.

Additionally, expanding the system to support multiple tailors and enabling better order management workflows will improve scalability and operational efficiency.

### Long-Term Vision

In the long term, the platform can evolve into a comprehensive digital tailoring ecosystem. This may include AI-based design recommendations, automated measurement detection using images, and integration with logistics services for delivery tracking.

The system can also be expanded to support large-scale deployment, allowing multiple tailoring businesses to use the platform. With continuous improvements, it has the potential to become a widely adopted solution for modernizing the tailoring industry.

# APPENDICES

## Appendix A – Terraform Code Snippets

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~>3.0"
    }
  }
}

provider "azurerm" {
  features {}
}

# Resource Group
resource "azurerm_resource_group" "marketplace" {
  name     = "rg-marketplace-${var.environment}"
  location = var.location
}

# Container Apps Environment
resource "azurerm_container_app_environment" "marketplace" {
  name                = "cae-marketplace-${var.environment}"
  location            = azurerm_resource_group.marketplace.location
  resource_group_name = azurerm_resource_group.marketplace.name
}

# PostgreSQL Flexible Server
resource "azurerm_postgresql_flexible_server" "marketplace" {
  name                   = "psql-marketplace-${var.environment}"
  resource_group_name    = azurerm_resource_group.marketplace.name
  location               = azurerm_resource_group.marketplace.location
  version                = "13"
  administrator_login    = var.db_admin_username
  administrator_password = var.db_admin_password
  zone                   = "1"

  storage_mb = 32768
  sku_name   = "B_Standard_B1ms"

  backup_retention_days = 7
}
```

```
resource "azurerm_postgresql_flexible_server_database" "marketplace" {
  name      = var.db_name
  server_id = azurerm_postgresql_flexible_server.marketplace.id
  collation = "en_US.utf8"
  charset   = "utf8"
}

# PostgreSQL Firewall Rule (Allow Azure Services)
resource "azurerm_postgresql_flexible_server_firewall_rule" "azure_services" {
  name             = "AllowAzureServices"
  server_id        = azurerm_postgresql_flexible_server.marketplace.id
  start_ip_address = "0.0.0.0"
  end_ip_address   = "0.0.0.0"
}

# Azure Storage Account for product images
resource "azurerm_storage_account" "marketplace_storage" {
  name                     = var.storage_account_name
  resource_group_name      = azurerm_resource_group.marketplace.name
  location                 = azurerm_resource_group.marketplace.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
  account_kind             = "StorageV2"
  access_tier              = "Hot"

  tags = {
    environment = var.environment
  }
}

# Storage Container for images
resource "azurerm_storage_container" "images" {
  name                  = var.storage_container_name
  storage_account_name  = azurerm_storage_account.marketplace_storage.name
  container_access_type = "blob" # Public access to blobs
}

# Static Web App created manually in Azure Portal
# Name: swa-marketplace-dev

# Container App
resource "azurerm_container_app" "marketplace_backend" {
  name                         = "ca-marketplace-backend-${var.environment}"
  container_app_environment_id = azurerm_container_app_environment.marketplace.id
```

*Figure 9. Main.tf code*

```
# Container App
resource "azurerm_container_app" "marketplace_backend" {
  name                         = "ca-marketplace-backend-${var.environment}"
  container_app_environment_id = azurerm_container_app_environment.marketplace.id
  resource_group_name          = azurerm_resource_group.marketplace.name
  revision_mode                = "Single"

  template {
    container {
      name   = "marketplace-backend"
      image  = "roshh4/marketplace-backend-alpine-amd64:${var.container_image_tag}"
      cpu    = 0.25
      memory = "0.5Gi"

      env {
        name  = "DB_HOST"
        value = azurerm_postgresql_flexible_server.marketplace.fqdn
      }

      env {
        name  = "DB_PORT"
        value = "5432"
      }

      env {
        name  = "DB_NAME"
        value = var.db_name
      }

      env {
        name  = "DB_USER"
        value = var.db_admin_username
      }

      env {
        name        = "DB_PASSWORD"
        secret_name = "db-password"
      }

      env {
        name  = "PORT"
        value = "8080"
      }
```

```
resource "azurerm_container_app" "marketplace_backend" {
  template {
    container {

      env {
        name  = "GIN_MODE"
        value = "release"
      }

      env {
        name  = "DB_SSLMODE"
        value = "require"
      }

      env {
        name  = "AZURE_STORAGE_ACCOUNT_NAME"
        value = var.storage_account_name
      }

      env {
        name        = "AZURE_STORAGE_CONNECTION_STRING"
        secret_name = "storage-connection-string"
      }

      env {
        name        = "GEMINI_API_KEY"
        secret_name = "gemini-api-key"
      }

      env {
        name  = "CONTENT_SAFETY_ENDPOINT"
        value = var.content_safety_endpoint
      }

      env {
        name        = "CONTENT_SAFETY_KEY"
        secret_name = "content-safety-key"
      }
    }

    min_replicas = 0
    max_replicas = 10
  }
```

*Figure 10. Main.tf code*

```tf
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~>3.0"
    }
  }
}

provider "azurerm" {
  features {}
}

# Resource Group
resource "azurerm_resource_group" "marketplace" {
  name     = "rg-marketplace-${var.environment}"
  location = var.location
}

# Container Apps Environment
resource "azurerm_container_app_environment" "marketplace" {
  name                = "cae-marketplace-${var.environment}"
  location            = azurerm_resource_group.marketplace.location
  resource_group_name = azurerm_resource_group.marketplace.name
}

# PostgreSQL Flexible Server
resource "azurerm_postgresql_flexible_server" "marketplace" {
  name                   = "psql-marketplace-${var.environment}"
  resource_group_name    = azurerm_resource_group.marketplace.name
  location               = azurerm_resource_group.marketplace.location
  version                = "13"
  administrator_login    = var.db_admin_username
  administrator_password = var.db_admin_password
  zone                   = "1"

  storage_mb = 32768
  sku_name   = "B_Standard_B1ms"

  backup_retention_days = 7
}
```

```tf
resource "azurerm_postgresql_flexible_server_database" "marketplace" {
  name      = var.db_name
  server_id = azurerm_postgresql_flexible_server.marketplace.id
  collation = "en_US.utf8"
  charset   = "utf8"
}

# PostgreSQL Firewall Rule (Allow Azure Services)
resource "azurerm_postgresql_flexible_server_firewall_rule" "azure_services" {
  name             = "AllowAzureServices"
  server_id        = azurerm_postgresql_flexible_server.marketplace.id
  start_ip_address = "0.0.0.0"
  end_ip_address   = "0.0.0.0"
}

# Azure Storage Account for product images
resource "azurerm_storage_account" "marketplace_storage" {
  name                     = var.storage_account_name
  resource_group_name      = azurerm_resource_group.marketplace.name
  location                 = azurerm_resource_group.marketplace.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
  account_kind             = "StorageV2"
  access_tier              = "Hot"

  tags = {
    environment = var.environment
  }
}

# Storage Container for images
resource "azurerm_storage_container" "images" {
  name                  = var.storage_container_name
  storage_account_name  = azurerm_storage_account.marketplace_storage.name
  container_access_type = "blob" # Public access to blobs
}

# Static Web App created manually in Azure Portal
# Name: swa-marketplace-dev

# Container App
resource "azurerm_container_app" "marketplace_backend" {
  name                         = "ca-marketplace-backend-${var.environment}"
  container_app_environment_id = azurerm_container_app_environment.marketplace.id
```

*Figure 11. Main.tf code*

[outputs.tf](#)

```tf
output "container_app_name" {
  description = "Name of the Container App"
  value       = azurerm_container_app.marketplace_backend.name
}

output "container_app_url" {
  description = "URL of the Container App"
  value       = "https://${azurerm_container_app.marketplace_backend.ingress[0].fqdn}"
}

output "container_app_environment_name" {
  description = "Name of the Container App Environment"
  value       = azurerm_container_app_environment.marketplace.name
}

# Static Web App outputs removed - created manually

output "database_host" {
  description = "PostgreSQL server hostname"
  value       = azurerm_postgresql_flexible_server.marketplace.fqdn
}

output "database_name" {
  description = "PostgreSQL database name"
  value       = azurerm_postgresql_flexible_server_database.marketplace.name
}

output "resource_group_name" {
  description = "Name of the resource group"
  value       = azurerm_resource_group.marketplace.name
}

output "storage_account_name" {
  description = "Name of the Azure Storage Account"
  value       = azurerm_storage_account.marketplace_storage.name
}

output "postgresql_server_name" {
  description = "Name of the PostgreSQL server"
  value       = azurerm_postgresql_flexible_server.marketplace.name
}

output "function_app_name" {
  description = "Name of the Azure Function App"
```

```tf
output "function_app_name" {
  description = "Name of the Azure Function App"
  value       = azurerm_linux_function_app.ai_description.name
}

output "function_app_url" {
  description = "URL of the Azure Function App"
  value       = "https://${azurerm_linux_function_app.ai_description.default_hostname}"
}

output "function_app_hostname" {
  description = "Hostname of the Azure Function App"
  value       = azurerm_linux_function_app.ai_description.default_hostname
}
```

*Figure 12. Outputs.tf code*

Variables.tf

```
variable "environment" {
  description = "Environment name (dev, staging, prod)"
  type        = string
  default     = "dev"
}

variable "location" {
  description = "Azure region for resources"
  type        = string
  default     = "Southeast Asia"
}

variable "db_admin_username" {
  description = "PostgreSQL administrator username"
  type        = string
  default     = "marketplace_admin"
}

variable "db_admin_password" {
  description = "PostgreSQL administrator password"
  type        = string
  sensitive   = true
}

variable "db_name" {
  description = "PostgreSQL database name"
  type        = string
  default     = "marketplace"
}

variable "storage_account_name" {
  description = "Azure Storage Account name"
  type        = string
}

variable "storage_container_name" {
  description = "Azure Storage Container name for images"
  type        = string
  default     = "images"
}

variable "container_image_tag" {
  description = "Docker image tag for the container"
  type        = string
```

```
variable "container_image_tag" {
  description = "Docker image tag for the container"
  type        = string
  default     = "latest"
}

variable "gemini_api_key" {
  description = "Gemini API key for AI description generation"
  type        = string
  sensitive   = true
}

variable "content_safety_endpoint" {
  description = "Azure Content Safety API endpoint"
  type        = string
  default     = "https://image-content-moderation.services.ai.azure.com"
}

variable "content_safety_key" {
  description = "Azure Content Safety API key"
  type        = string
  sensitive   = true
}
```

*Figure 13. variables.tf code*

# Appendix B – Pipeline YAMLs

c

```yaml
name: CI Build Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-backend:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Login to Docker Hub
      uses: docker/login-action@v3
      with:
        username: ${{ secrets.DOCKERHUB_USERNAME }}
        password: ${{ secrets.DOCKERHUB_TOKEN }}

    - name: Build and push backend
      uses: docker/build-push-action@v5
      with:
        context: ./backend
        file: ./backend/Dockerfile
        push: true
        tags: |
          alphastar59/marketplace-backend-alpine-amd64:latest
        platforms: linux/amd64

  build-frontend:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
```

```yaml
  build-frontend:

    steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: '18'
        cache: 'npm'
        cache-dependency-path: './frontend/package-lock.json'

    - name: Install dependencies
      run: |
        cd frontend
        npm ci

    - name: Build frontend
      run: |
        cd frontend
        npm run build

    - name: Upload build artifacts
      uses: actions/upload-artifact@v4
      with:
        name: frontend-build
        path: frontend/dist/
```

*Figure 14. Main Branch Workflow file*

```yaml
jobs:
  deploy-backend:
    - name: Get Backend URL
      run: |

  deploy-frontend:
    runs-on: ubuntu-latest
    needs: deploy-backend

    steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: '18'
        cache: 'npm'
        cache-dependency-path: './frontend/package-lock.json'

    - name: Install dependencies
      run: |
        cd frontend
        npm ci

    - name: Build frontend with backend URL
      run: |
        cd frontend
        echo "VITE_API_URL=${{ needs.deploy-backend.outputs.backend-url }}" > .env.production
        npm run build

    - name: Deploy to Azure Static Web Apps
      uses: Azure/static-web-apps-deploy@v1
      with:
        azure_static_web_apps_api_token: ${{ secrets.AZURE_STATIC_WEB_APPS_API_TOKEN }}
        repo_token: ${{ secrets.GITHUB_TOKEN }}
        action: "upload"
        app_location: "./frontend"
        output_location: "dist"
```

**Figure 15. Prod Branch Workflow file**

# Appendix C – Screenshots (Deployments, AI Integration, Security Scans)

**Successful ci/cd run:**

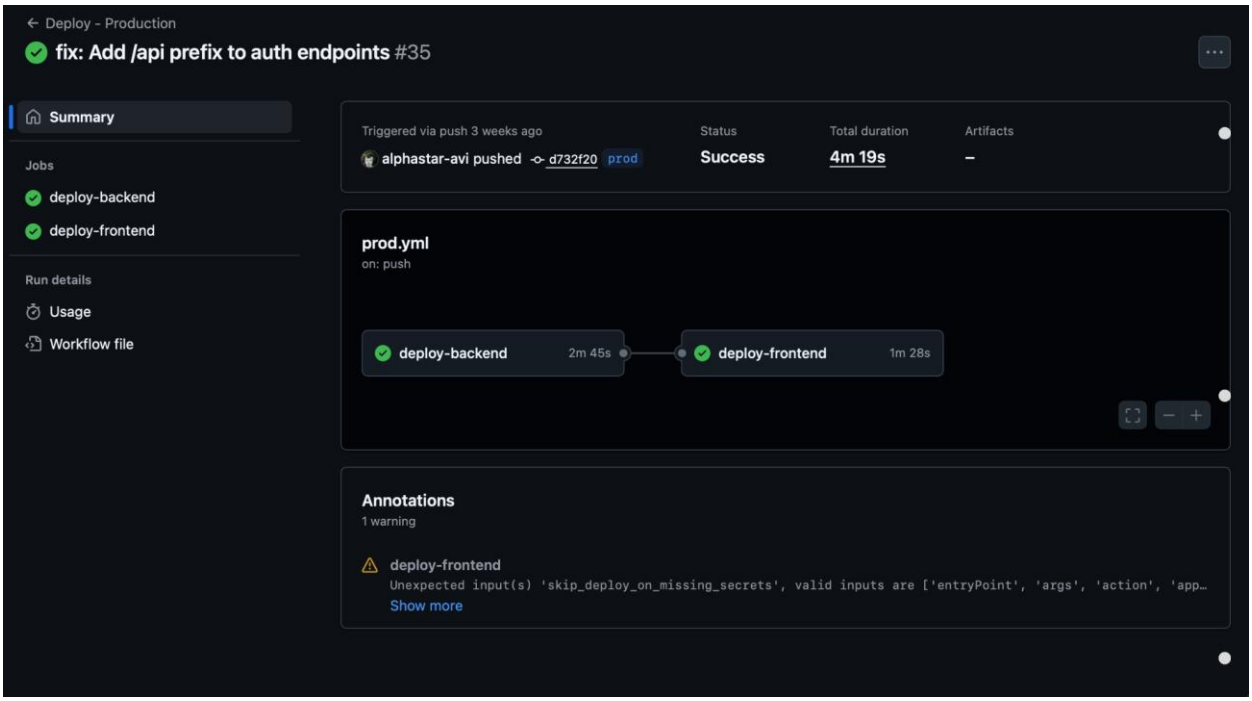


*Figure 16. Workflow Showing Frontend and Backend Successfully Deployed*

**Integrated AI services:**

1. **Azure content moderation**

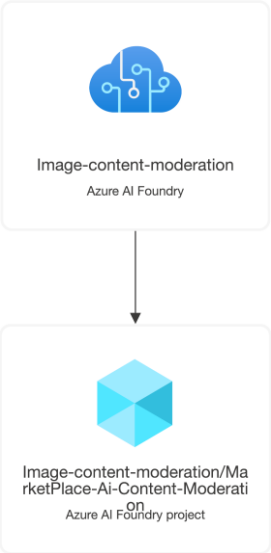


*Figure 17. Azure content moderation*

## 2. Azure Container Apps

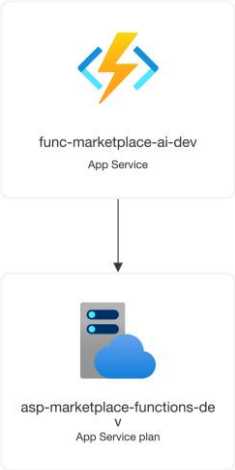


*Figure 18. Description Generation with gemini vision pro*

## Deployment resource visualizer

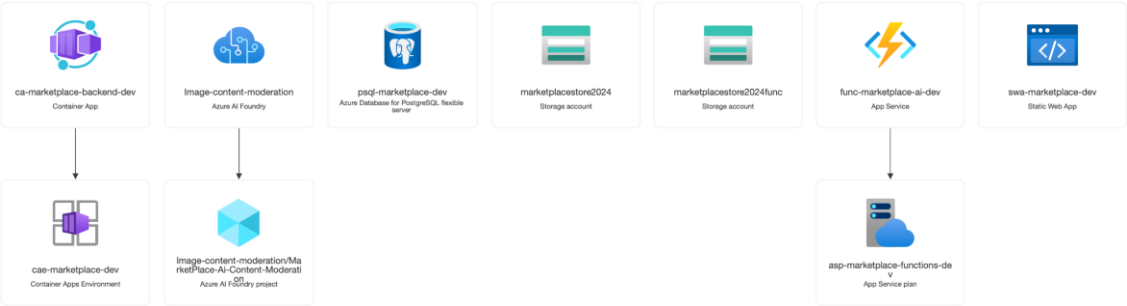


*Figure 19. Network Visualizer*

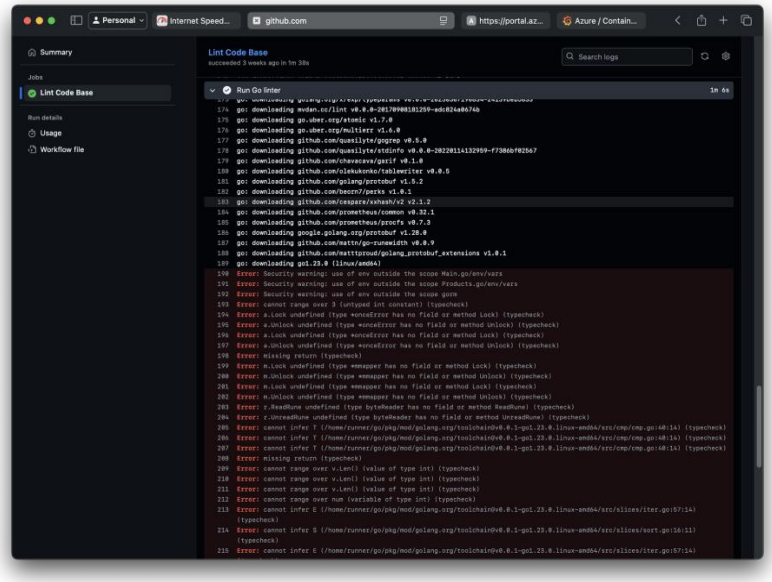


*Figure 20. Link Scans*